Air Quality Forecasting Model for Seoul, Korea

Kuanyu Chen

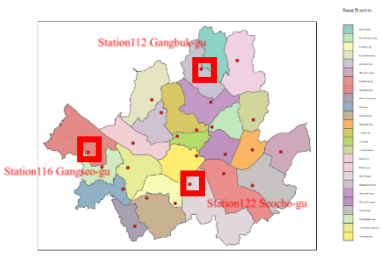
Introduction and data description

Air quality is an issue of concern to a wide range of people in modern society. Pollutants such as particulate matter (PM) come from a variety of sources, either from human activity or from natural sources.

According to the World Health Organisation (WHO, 2022), almost 99% of the world's population in 2019 was living in areas where air quality exceeded the required quality standards. PM is more hazardous than other pollutants because the particles are smaller and are more dangerous to humans in the gas exchange portion of the lungs (Dockery et al., 1994).

Therefore, PM10 and PM2.5 are two pollutants that have received much attention in air quality studies. Research on air pollution in six Asian cities from 2001 to 2004 (N.T. Kim Oanh et al., 2006) studied the spatial and temporal correlation between PM10 and PM2.5 and found that the concentrations of these two pollutants were higher during the dry season. Another study (Chow et al., 2006) was conducted in central California and found that PM2.5 data decreased rapidly at higher altitudes due to topography, and those high concentrations occurred between November and January. This study obtained data from air quality measurement stations in Seoul, Korea, for a total of 25 stations for each hour of each day for the three years from 2017 to 2019 through the Kaggle (Bappe, 2020). The data include six pollutants (SO2, NO2, O3, CO, PM10, PM2.5). In this report, the data will only focus on suspended particulate matter (PM10 and PM2.5), considering the significant risk of these fine particulate matter to humans. The ARIMA model will be used to predict air quality, with the first two years as the training set and the last year as the test set. This study found that air pollution in Seoul may include other factors such as altitude, human activity, or urban land use. Seasonal influences are observed to have a more significant effect on air quality.

The distribution of the data in terms of geospatial location is then shown in Figure 1 below.

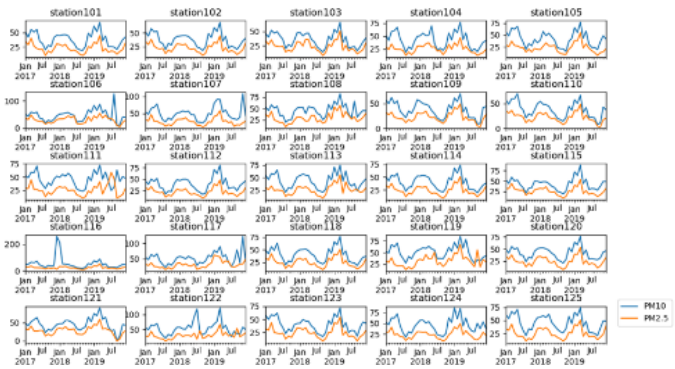


(Figure 1. Location of air quality measurement stations in Seoul)

Each administrative district in Seoul has a station for air quality monitoring, which is evenly distributed. In this study, the three stations are selected for prediction using an ARIMA model. Considering the population distribution between the three stations, Gangbuk-gu district has a population density of 35,000/km^2 in 2010, which is the highest in Seoul, and Seocho-gu has a high proportion of residential areas in the urban planning part (Seoul solution, 2021), the above-mentioned air pollution risks to human beings have led to a focus on more populated and residential areas. Gangseo-gu (station116) was chosen because it has a different distribution of values, and therefore modelling will be attempted to see if ARIMA can be used to predict future trends in this area.

Exploratory spatio-temporal data analysis

Firstly, the units of analysis will be determined before looking at the general distribution of the data. In view of the high level of detail of the data, which includes hourly measurement, it was first collated into a monthly average before visualising the data distribution.

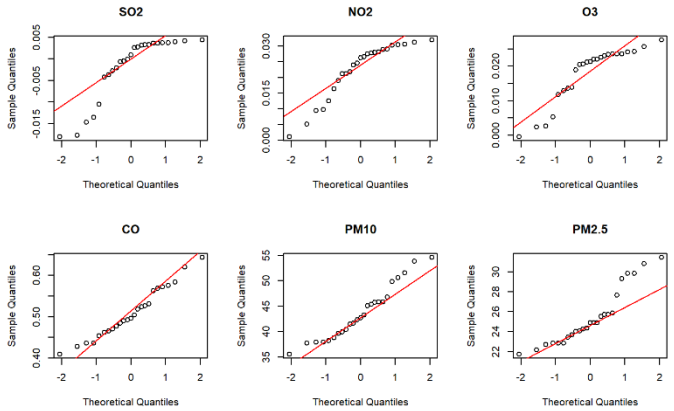


(Figure 2. Trends in PM10 and PM2.5 data for all stations)

As can be seen from the graph above, PM10 is higher than PM2.5 at almost all times. Furthermore, many of the stations have similar pollutant peaks, for example, higher values in winter (January), so we can tentatively judge the data to be seasonal in nature.

The more unusual of all stations is station 116, where PM2.5 values do not even vary significantly, except for a peak in PM10 in 2018, which is relatively stable compared to other stations, and which is also predicted in this report to see if the model can match future trends when comparing different data characteristics. In another section, the quantile-quantile (QQ) plot for the six pollutants in Seoul, without considering the different stations,

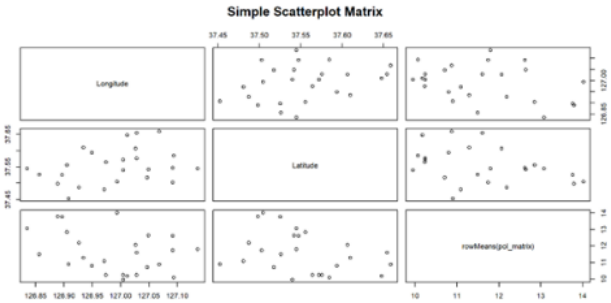
is shown in Figure 3 below.



(Figure 3. Quantile-quantile (QQ) plot for the six pollutants in Seoul)

Data points that are out of the linear line and trend horizontally represent the peaks of the data distribution. The distribution of CO roughly conforms to a normal distribution.

The next step is to look at the state of the data numerically to determine if there is a geospatial or temporal correlation. The correlation between the pollutants and the latitude and longitude of the stations will be further observed in Figure 4 and no significant trends can be observed.

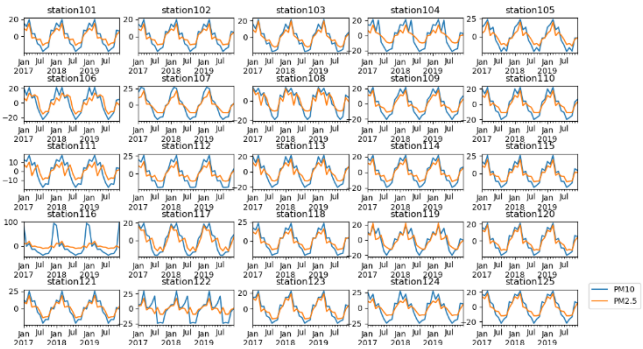


(Figure 4. Scattered matrix of pollutants and station latitude and longitude)

Methodology and results

Before starting with the model predictions, it is necessary to understand whether the pattern of the data is likely to be stable or not. In the above analysis, the trend is rather volatile and not stable. On the other hand, there may be seasonal trends, so this part of the model starts with a seasonal decomposition using moving averages. As the data are fluctuating on an annual basis, the period is set to 12.

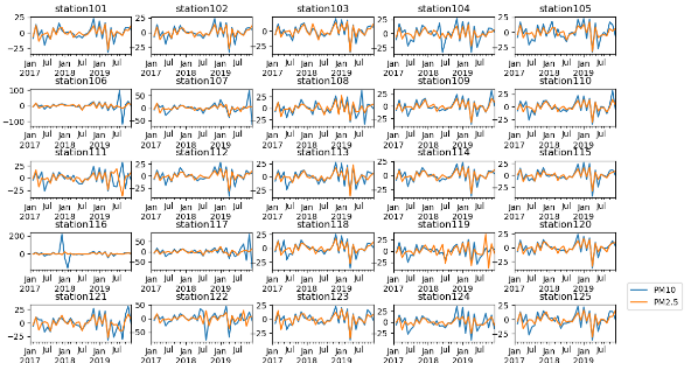
Figure 5 below shows that there is a seasonal trend for both pollutants at almost all stations. Furthermore, it is worth discussing that, in line with previous studies in the literature, higher concentrations are observed during the winter months.



(Figure 5. Seasonal decomposition of PM10 and PM2.5 values for all stations)

In addition, one of the stations to be discussed in this report (station 116) has a relatively flat seasonal trend in pm2.5 data compared to the other stations. Therefore, it is possible to focus on the modeling of pm2.5 at station 116.

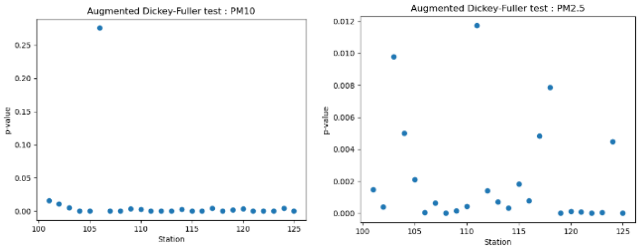
After observing the seasonal trends, the data should be stabilised. First, the data are first smoothed using the non-seasonal differencing to smooth the data. As can be seen in Figure 6 below, most of the data has been removed from the obvious seasonal fluctuations, but some of the stations show a larger trend change in early 2019.



(Figure 6. Non-seasonal differencing in PM10 and PM2.5 values for all stations)

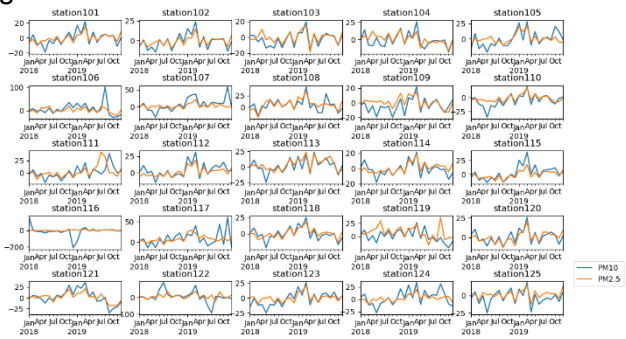
Therefore, a unit root test is used to determine if the data are stationary. The Augmented Dickey-Fuller test is one of the most common methods (Rathore et al., 2022). If the test rejects the null hypothesis (p-value less than 0.05), then the data are stationary.

Figure 7 below shows the p-value of these two pollutants at each station. In terms of PM10, most of the stations show a stable condition, with only station106 being greater than 0.05, which is a non-stationary condition. PM2.5 is stable after differencing; therefore a single differencing is sufficient for PM2.5 prediction.



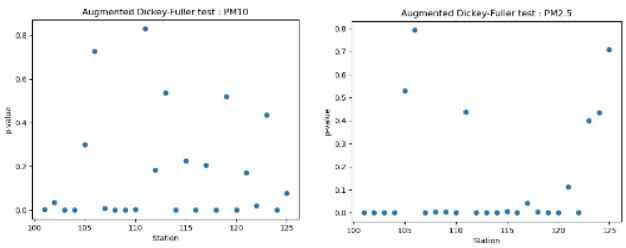
(Figure 7. ADF stability test for PM10 and PM2.5(non-seasonal differencing))

However, given the seasonal trend of air pollutant data in Seoul, it is necessary to understand the seasonal differences in the construction of the ARIMA model. The results after observing the seasonal differences for each station are shown in Figure 8 below.



(Figure 8. Seasonal differencing in PM10 and PM2.5 values for all stations)

As can be observed in Figure 8, after a seasonal differencing, the trend for the two pollutants does not appear to be more significant than the normal differencing, but further analysis is still required. Therefore, it is also appropriate to use the Augmented Dickey-Fuller test to see if the data for the two pollutants are stable after differencing.



(Figure 9. ADF stability test for PM10 and PM2.5(seasonal differencing))

First, it can be seen that the PM10 results after only one seasonal difference show that there are a number of stations that are unstable, i.e. stations 105, 106, 111, 112, 113, 115, 117, 119, 121, 123, 125. In building the models for these stations, a differencing process will be required, followed by a seasonal differencing process, to ensure that the data are stable.

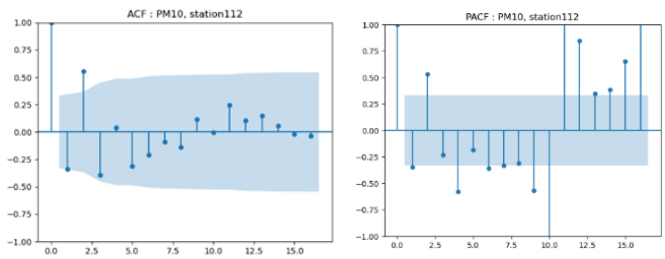
For PM2.5, the results after seasonal differencing show that some stations are also non-stationary after one seasonal differencing, e.g. stations 105,106,111,121,125, and there is less non-stationary data for PM2.5 after differencing than for PM10.

Considering the stations 112, 116,122 that this study aims to focus on, once the seasonal patterns have been removed by differencing, an appropriate ARIMA model can be constructed to simulate the stationary time series. The order of the ARIMA model can be determined from the ACF and PACF plots of the differenced data. When constructing a model for data with seasonal trends, it is necessary to examine the ACF and PACF plots for non-seasonal differenced, as well as seasonal differenced data to determine the optimum values for (p, d, q) and (P, D, Q).

To determine the parameters, the ACF and PACF plots for non-seasonal and seasonal differences are first observed and the best combination of parameters is selected using the auto.ARIMA function respectively. After obtaining the (p,d,q) or (P,D,Q) values (for the seasonal difference model), the parameters are then checked for autocorrelation between the residuals. If the test is not passed (meaning there is significant autocorrelation), then there is more information in the residuals that has not been considered in the model and is not suitable for modelling. Finally, the differences between the two (manual observation and automatic selection) are considered and discussed.

First, in the following section, the value of station112 is noted to determine the parameter settings.

- **Non-seasonal differencing:**
  - PM10:



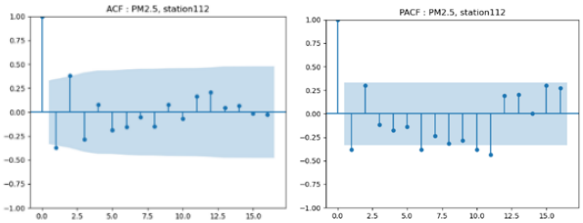
(Figure 10. The ACF versus PACF plot for PM10 (non-seasonal differencing))

First, the significant lag in the ACF plot gives a general idea of the value of q. In this case, the ACF is noticeable at lags 1 and 2, so q can be set to 2.

The value of p is determined by the significant

lag in the PACF plot. In this case, the PACF is apparent at lags 1, 2, 4, 6, 7, 9, 10 to 16. Based on this, it may be possible to start with a value of p = 16 and gradually reduce it to find the minimum value of p that produces a good fit. Furthermore, significant spikes in the PACF plot that exceed lag 5 may indicate the need for a higher order of autoregression.

- PM2.5:

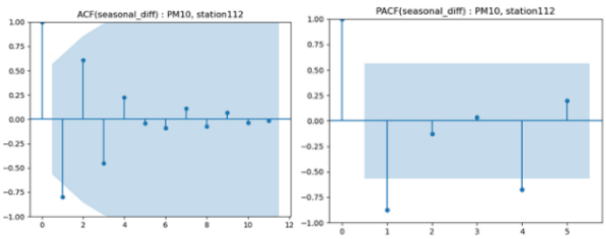


(Figure 11. The ACF versus PACF plot for PM2.5 (non-seasonal differencing))

Observing the PM2.5 data, the parameter settings can be inferred in the similar way, with p being set to12 or less and q to 2.It is worth noting that when doing an ARIMA model requires experimenting with different combinations of (p, d, q) and using model evaluation techniques (e.g. AIC) to select the best model for the data. In contrast, this study will allow the model to automatically generate the lowest AIC parameter combinations directly through auto.ARIMA in the next section, for which a predictive model will be built once the parameter testing is completed.

- **Seasonal differencing:**
  - PM10:

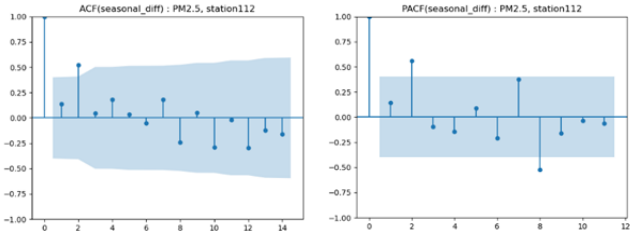
As mentioned above, the station112 PM10 data was non-stationary after seasonal differencing. Therefore, before drawing ACF and PACF plots, a first-order differencing was performed, followed by seasonal differencing.



(Figure 12. The ACF versus PACF plot for PM10 (seasonal differencing))

A similar procedure will be employed, with Q=1 tentatively determined from the ACF plot, and P=4 initially inferred from the PACF plot. Note that in this context, P, D, and Q are all capitalized to differentiate them as seasonal parameters.

- PM2.5



(Figure 13. The ACF versus PACF plot for PM2.5 (seasonal differencing))

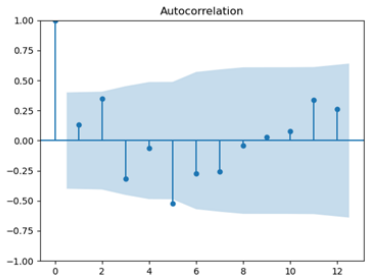
Similarly, for the PM2.5 data, the ACF plot suggests a tentative value of Q=2, while the PACF plot initially indicates a value of P=8.

After completing the preliminary parameter analysis, the auto.ARIMA function in Python will be applied to input approximate parameters and identify the model with the lowest AIC value that best fits the future trend after validation.

- **Non-seasonal ARIMA models**
  - PM10:

Starting with PM10, an attempt will be made to build a non-seasonal model. Hence, the parameters (p,d,q) will be input with starting and maximum values. The approximate range obtained from the ACF and PACF in the previous step will be used as input to the auto.ARIMA function. Upon completion, the output suggests that the optimal parameter setting is (1,0,0), with an AIC value of 175.969. Next, the residuals will be tested for stationarity.

As shown in Figure14, autocorrelation has not been completely removed; hence, the idea of building a non-seasonal PM10 ARIMA model will be temporarily discarded.

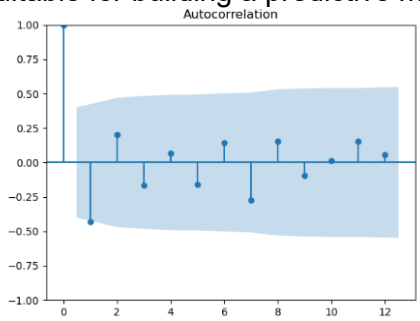


(Figure 14. Non-seasonal model residual autocorrelation plot (PM10))

- PM2.5:

Next, a non-seasonal model will be attempted on the PM2.5 data, following the same

process as with PM10. The optimal parameter setting suggested by the model output is (0,1,0). Similarly, the autocorrelation plot of the residuals will be examined to determine if it is suitable for building a predictive model.



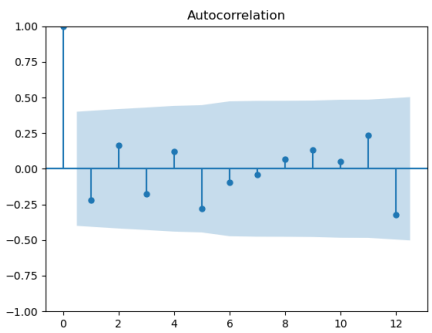
(Figure 15. Non-seasonal model residual autocorrelation plot (PM2.5))

As seen in Figure 15, the results are the same as with PM10, and the autocorrelation of the residuals has not been removed.

After completing the non-seasonal models, the next step is to attempt to build seasonal models. Following the same procedure as above, the initial parameter ranges will be determined through ACF and PACF, and then input into the parameters of auto.ARIMA. The residuals will then be checked through a diagnostic test to confirm whether the model can be used to predict future trends.

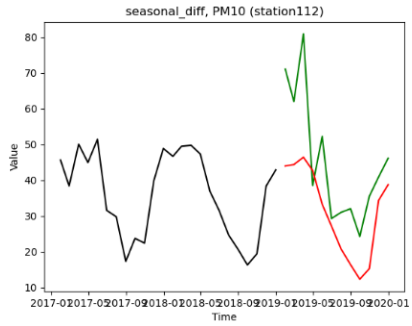
- **Seasonal ARIMA models**
  - PM10:

The output information after completion indicates that the optimal parameter setting is (1,1,0)x(0,1,0), with an AIC value of 76.616. Next, it will be verified whether the residuals pass the test. From Figure 16 below, it can be seen that the autocorrelation of the residuals has been completely removed.



(Figure 16. Seasonal model residual autocorrelation plot (PM10))

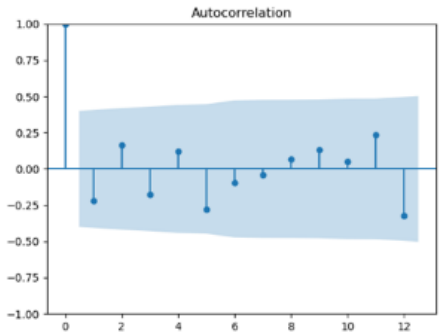
Therefore, this parameter can be used to build a prediction model. As shown in Figure 17, the red trend line represents the predicted result. The overall trend conforms to the future, but the curve is too smooth and cannot accurately predict the peaks and changes of each period.



(Figure 17. Future Trends with Seasonal Models (PM10))

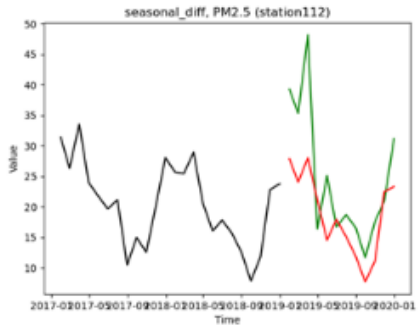
- PM2.5:

Similarly, after completing the output of the parameter for PM2.5, auto.ARIMA suggested the best settings to be (2,1,0)x(0,1,0). Also, as before, the AIC value can be obtained from the .summary() output, which is 61.885 in this case. From the figure 18 below, it can be seen that autocorrelation has also been removed.



(Figure 18. Seasonal model residual autocorrelation plot (PM2.5))

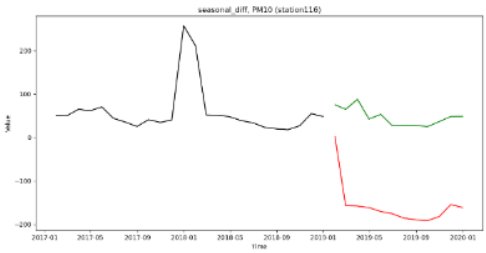
After creating the forecast for the last year, it is plotted in Figure 19. It can be observed that compared to the trend forecast for PM10, the PM2.5 model seems to have more pronounced peaks and better matches the actual future values.



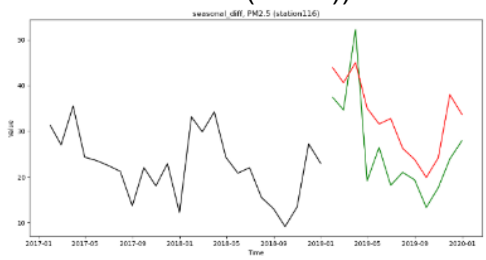
(Figure 19. Future Trends with Seasonal Models (PM2.5))

The same steps were taken to establish the future trend prediction of PM2.5 for stations 116 and 122. The auto.ARIMA was confirmed

to have passed the test for parameter combination before making the final prediction for the last year. Firstly, for station 116, it was evident that the prediction for PM10 was not ideal. Both parameter combinations were (0,1,0), indicating that there were no autoregressive terms (p=0) and no moving average terms (q=0) in the ARIMA model. The trend of the PM10 raw data (only one peak) could potentially affect the prediction results, possibly due to the existence of outliers or anomalous data. On the other hand, PM2.5 relatively followed the future trend, even though its raw data trend for this station was smoother compared to other stations.

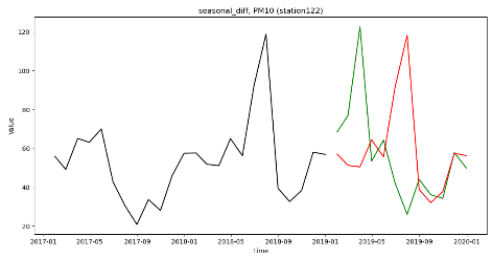


(Figure 20. Future Trends with Seasonal Models (PM10))

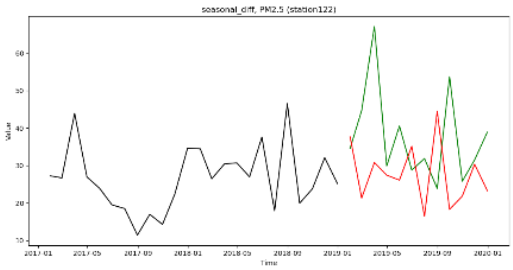


(Figure 21. Future Trends with Seasonal Models (PM2.5))

In the case of station 122, it can be observed that the predicted values of PM10 seem to have a delay. Despite attempting to increase the order of the model parameter, the best combination is still (0,1,0). Therefore, it is possible that the model overfits historical data, leading to such results. As for the PM2.5 prediction, it is more in line with the trend at the end of 2019, but it did not successfully predict the peak value in mid-2019.



(Figure 22. Future Trends with Seasonal Models (PM10))



(Figure 23. Future Trends with Seasonal Models (PM2.5))

Discussion and conclusions

In this study, the ARIMA model was used to predict PM10 and PM2.5 concentrations at three selected stations in Seoul. Station 112 exhibited the best performance, with a broadly aligned trend, albeit unable to predict each peak precisely. Station 122 also demonstrated general conformity to the trend, but with a tendency for delayed peaks in PM10, potentially due to overfitting of the model to historical data.

Conversely, Station 116 featured significant outliers in the observed values, resulting in inaccurate final results. Overall, the suitability of PM2.5 for the ARIMA model was higher than that of PM10, likely because PM10 concentrations are influenced by factors not accounted for in this study, such as weather conditions, traffic patterns, or industrial emissions. As such, air quality data may feature complex seasonal patterns beyond the scope of seasonal ARIMA models.

To enhance the accuracy of ARIMA models for air quality prediction, several considerations should be made.

Firstly, external variables such as weather data, traffic volumes, and industrial emissions should be incorporated into the model to capture the non-stationarity of air quality data and improve model accuracy. Secondly, more advanced time series models that capture complex seasonal patterns and relationships between variables could be implemented. Alternatively, machine learning techniques such as neural networks or support vector machines could be utilized to model the non-linear relationship between air quality and external variables.

Reference:

1. Bappe (2020) Air pollution in Seoul, Kaggle. Available at: https://www.kaggle.com/datasets/bappekim/air-pollution-in-seoul (Accessed: 13 January, 2023).

2. World Health Organization(2022).Ambient (outdoor) air pollution. Available at: https://www.who.int/news-room/fact-sheets/detail/ambient-(outdoor)-air-quality-and-health (Accessed: 1 March, 2023).

3. Dockery, D. W., and Pope, C. A. (1994). “Acute respiratory effects of particulate air pollution,” *Annual Review of Public Health*, 15(1), pp. 107–132. Available at: https://doi.org/10.1146/annurev.pu.15.050194.000543.

4. Oanh, N. K., Upadhyay, N., Zhuang, Y. H., Hao, Z. P., Murthy, D. V. S., Lestari, P., ... & Lindgren, E. S. (2006). “Particulate Air Pollution in six Asian cities: Spatial and temporal distributions, and associated sources,” *Atmospheric Environment*, 40(18), pp. 3367–3380. Available at: https://doi.org/10.1016/j.atmosenv.2006.01.050.

5. Chow, J. C., Chen, L. W. A., Watson, J. G., Lowenthal, D. H., Magliano, K. A., Turkiewicz, K., & Lehrman, D. E. (2006) “PM2.5chemical composition and spatiotemporal variability during the California regional PM10/pm2.5air quality study (CRPAQS),” *Journal of Geophysical Research: Atmospheres*, 111(D10). Available at: https://doi.org/10.1029/2005jd006457.

6. Seoul solution (2021) The social maps of Seoul, 서울정책아카이브 Seoul Solution. Available at: https://seoulsolution.kr/en/content/3324 (Accessed: 9 March, 2023).

7. Rathore, R.K. et al. (2022) “Real-world model for bitcoin price prediction,” *Information Processing & Management*, 59(4), p. 102968. Available at: https://doi.org/10.1016/j.ipm.2022.102968.

Code Transcription (R & Python)

ESDA in R:

```
#1.1 Import the data
library("xlsx")
library(GMCM)
library(Hmisc)
library(psych)
library(ggplot2)
library(dplyr)
library(scatterplot3d)
library(plot3D)
library(rgl)
library(maptools)
library(lattice)
library(spdep)
library(sp)
library(rgdal)
library(tmap)
library(sf)
library(ggplot2)
library(gridExtra)
library(gstat)
library(OpenStreetMap)
library(spacetime)
library(raster)
setwd("D:/CEGE/★TERM_2/0042 STDM/AirPollutionSeoul/")
avg_year <- read.xlsx("stations_avgperyear.xlsx",1, header=TRUE)
item_info <- read.csv("Measurement_item_info.csv")
station_info <- read.csv("Measurement_station_info.csv")
sl_map <- readOGR(dsn="D:/CEGE/★TERM_2/0042 STDM/AirPollutionSeoul",
layer="seoul_municipalities")
stations <- st_as_sf(avg_year, coords = c('Longitude', 'Latitude'))
colnames(avg_year)[1] <- as.character("station")
pol_matrix<-data.matrix(avg_year[,4:ncol(avg_year)])

#1.2 Examining non spatio-temporal data characteristics
mu = colMeans(pol_matrix) #mean of each pollutants in each year
station_mean = rowMeans(pol_matrix)
Sds <- apply(pol_matrix,2,sd) #standard deviation
multi.hist(pol_matrix,density=FALSE,global=FALSE)
labels <- colnames(pol_matrix)
```

```
#QQ plot
par(mfrow=c(2,3))
for (i in 1:ncol(pol_matrix)){
   qqnorm(pol_matrix[, i], main = labels[i])
   qqline(pol_matrix[, i], col="red")
}
# Average pollution values for each station (scatter plot)
pairs(~Longitude+Latitude+rowMeans(pol_matrix),data=avg_year,
        main="Simple Scatterplot Matrix") #Pair Plot
scatterplot3d(x=avg_year$Longitude, y=avg_year$Latitude, z=rowMeans(pol_matrix))

# Comparison of pollutants at each station (scatter plot)
SO2 = pol_matrix[,1]
NO2 = pol_matrix[,2]
O3 = pol_matrix[,3]
CO = pol_matrix[,4]
PM10 = pol_matrix[,5]
PM2.5 = pol_matrix[,6]
pairs(~Longitude+Latitude+SO2,data=avg_year,
        main="SO2 - Simple Scatterplot Matrix") #Pair Plot
scatterplot3d(x=avg_year$Longitude, y=avg_year$Latitude, z=SO2)
pairs(~Longitude+Latitude+NO2,data=avg_year,
        main="NO2 - Simple Scatterplot Matrix") #Pair Plot
scatterplot3d(x=avg_year$Longitude, y=avg_year$Latitude, z=NO2)
pairs(~Longitude+Latitude+O3,data=avg_year,
        main="O3 - Simple Scatterplot Matrix") #Pair Plot
scatterplot3d(x=avg_year$Longitude, y=avg_year$Latitude, z=O3)
pairs(~Longitude+Latitude+CO,data=avg_year,
        main="CO - Simple Scatterplot Matrix") #Pair Plot
scatterplot3d(x=avg_year$Longitude, y=avg_year$Latitude, z=CO)
pairs(~Longitude+Latitude+PM10,data=avg_year,
        main="PM10 - Simple Scatterplot Matrix") #Pair Plot
scatterplot3d(x=avg_year$Longitude, y=avg_year$Latitude, z=PM10)
pairs(~Longitude+Latitude+PM2.5,data=avg_year,
        main="PM2.5 - Simple Scatterplot Matrix") #Pair Plot
scatterplot3d(x=avg_year$Longitude, y=avg_year$Latitude, z=PM2.5)


# plot stations' location
sf::sf_use_s2(FALSE)
tm_shape(sl_map)+
```

```
  tm_polygons("SIG_ENG_NM",title="Seoul Districts")+
  tm_legend(legend.outside = TRUE,legend.text.size=0.5)+
  tm_fill()+
tm_shape(stations)+
  tm_bubbles(col = "red", scale = .5)

#plot each pollutant in each station within these 3 yrs
sf::sf_use_s2(FALSE)
p1 <- tm_shape(sl_map)+
  tm_borders("black")+
  tm_shape(stations)+
  tm_dots("SO2", palette = "-Spectral", auto.palette.mapping = FALSE,size =
0.5,breaks=c(0,0.02,0.05,0.15,1))+
  tm_legend(legend.outside = TRUE)
p2 <- tm_shape(sl_map)+
  tm_borders("black")+
  tm_shape(stations)+
  tm_dots("NO2", palette = "-Spectral", auto.palette.mapping = FALSE,size =
0.5,breaks=c(0,0.03,0.06,0.2,2))+
  tm_legend(legend.outside = TRUE)
p3 <- tm_shape(sl_map)+
  tm_borders("black")+
  tm_shape(stations)+
  tm_dots("O3", palette = "-Spectral", auto.palette.mapping = FALSE,size =
0.5,breaks=c(0,0.03,0.09,0.15,0.5))+
  tm_legend(legend.outside = TRUE)
p4 <- tm_shape(sl_map)+
  tm_borders("black")+
  tm_shape(stations)+
  tm_dots("CO", palette = "-Spectral", auto.palette.mapping = FALSE,size = 0.5,breaks=c(0,2,9,15,50))+
  tm_legend(legend.outside = TRUE)
p5 <- tm_shape(sl_map)+
  tm_borders("black")+
  tm_shape(stations)+
  tm_dots("PM10", palette = "-Spectral", auto.palette.mapping = FALSE,size =
0.5,breaks=c(0,30,80,150,600))+
  tm_legend(legend.outside = TRUE)
p6 <- tm_shape(sl_map)+
  tm_borders("black")+
  tm_shape(stations)+
  tm_dots("PM2.5", palette = "-Spectral", auto.palette.mapping = FALSE,size =
```

```
0.5,breaks=c(0,15,35,75,500))+
  tm_legend(legend.outside = TRUE)
#tmap_arrange(p1)
tmap_arrange(p1,p2,p3,p4,p5,nrow = 2) # all measurements were in good condition

sf::sf_use_s2(FALSE)
p1 <- tm_shape(sl_map)+
  tm_borders("black")+
  tm_shape(stations)+
  tm_dots("SO2", palette = "-Spectral", auto.palette.mapping = FALSE,size = 0.5)+
  tm_legend(legend.outside = TRUE)
p2 <- tm_shape(sl_map)+
  tm_borders("black")+
  tm_shape(stations)+
  tm_dots("NO2", palette = "-Spectral", auto.palette.mapping = FALSE,size = 0.5)+
  tm_legend(legend.outside = TRUE)
p3 <- tm_shape(sl_map)+
  tm_borders("black")+
  tm_shape(stations)+
  tm_dots("O3", palette = "-Spectral", auto.palette.mapping = FALSE,size = 0.5)+
  tm_legend(legend.outside = TRUE)
p4 <- tm_shape(sl_map)+
  tm_borders("black")+
  tm_shape(stations)+
  tm_dots("CO", palette = "-Spectral", auto.palette.mapping = FALSE,size = 0.5)+
  tm_legend(legend.outside = TRUE)
p5 <- tm_shape(sl_map)+
  tm_borders("black")+
  tm_shape(stations)+
  tm_dots("PM10", palette = "-Spectral", auto.palette.mapping = FALSE,size = 0.5)+
  tm_legend(legend.outside = TRUE)
p6 <- tm_shape(sl_map)+
  tm_borders("black")+
  tm_shape(stations)+
  tm_dots("PM2.5", palette = "-Spectral", auto.palette.mapping = FALSE,size = 0.5)+
  tm_legend(legend.outside = TRUE)
tmap_arrange(p1,p2,p3,p4,p5,nrow = 2)
```

ESDA & ARIMA model in python:

```
import csv
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.stats
import statsmodels.api as sm
from statsmodels.tsa.stattools import adfuller
from matplotlib.pylab import style
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.seasonal import seasonal_decompose
import pmdarima as pm
from pmdarima import model_selection
from sklearn.metrics import mean_squared_error
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.tsa.statespace.tools import diff
from statsmodels.tsa.seasonal import seasonal_decompose


# Open CSV file
with open('Measurement_summary_01.csv', newline='') as csvfile:
    # Reading the contents of a CSV file
    rows = csv.DictReader(csvfile)

df = pd.read_csv('Measurement_summary_01.csv',index_col=0,parse_dates=[0])

# Create monthly data for different station
d = {}
for i in range(101,126):
    data_query = df.loc[df['Station code'] == i]
    filename="df" + str(i)
    d[filename] = data_query[['SO2','NO2','O3','CO','PM10','PM2.5']].resample("M").mean()

# Define the number of rows and columns of the sub diagram
# Plot the month data for each station (all pollutants)
nrow=5
ncol=5
fig, axes = plt.subplots(nrow, ncol)
# The plot counter starts with the first station number (101)
count=101
```

```
for r in range(nrow):
    for c in range(ncol):
        ax = d['df'+str(count)].plot(ax=axes[r,c],legend=None,title='station'+str(count))
        ax.set(xlabel=None)
        count+=1
plt.legend(bbox_to_anchor=(1.7,1.7))
plt.close()
plt.close(fig)
plt.show()

# Observe only pm10 and pm2.5 data
d2 = {}
d2_PM10 = {}
d2_PM25 = {}
for i in range(101,126):
  data_query = df.loc[df['Station code'] == i]
  filename="df" + str(i)
  d2[filename] = data_query[['PM10','PM2.5']].resample("M").mean()
  d2_PM10[filename] = data_query['PM10'].resample("M").mean()
  d2_PM25[filename] = data_query['PM2.5'].resample("M").mean()

# Plot monthly data for each station and plot seasonal trends (pm10 and pm2.5 only)
# Seasonal decomposition codes are referenced below
# Reference: https://machinelearningmastery.com/decompose-time-series-data-trend-seasonality/
nrow=5
ncol=5
fig, axes = plt.subplots(nrow, ncol)
count=101
for r in range(nrow):
    for c in range(ncol):
        measure_pm10 = d2_PM10['df'+str(count)]
        ssm_PM10_diff = seasonal_decompose(measure_pm10, period=12)
        ss_PM10_diff = ssm_PM10_diff.seasonal
        ss_PM10_diff.plot(ax=axes[r,c],legend=None,title='station'+str(count),label='PM10',xlabel = '')
        ax = d2['df'+str(count)].plot(ax=axes[r,c],legend=None,title='station'+str(count))
        ax.set(xlabel=None)

        measure_pm25 = d2_PM25['df'+str(count)]
        ssm_PM25_diff = seasonal_decompose(measure_pm25, period=12)
        ss_PM25_diff = ssm_PM25_diff.seasonal
        ss_PM25_diff.plot(ax=axes[r,c],legend=None,title='station'+str(count),label='PM2.5',xlabel = '')
```

```
        count+=1
plt.legend(bbox_to_anchor=(1.7,1.3))
plt.close()
plt.close(fig)
plt.show()



# Splitting into training and test data sets
nrow=5
ncol=5
count=101
PM10_train = {} # PM10 data for each station in 2017 to 2018
PM10_test = {} # PM10 data for each station in 2019
PM25_train = {} # PM2.5 data for each station in 2017 to 2018
PM25_test = {} # PM2.5 data for each station in 2019
for r in range(nrow):
    for c in range(ncol):
        # splitting pm10 data
        data = d2_PM10['df'+str(count)]
        d2_PM10_train = data[:24]
        d2_PM10_test = data[-12:]
        PM10_train["PM10_station"+str(count)] = d2_PM10_train
        PM10_test["PM10_station"+str(count)] = d2_PM10_test
        # a3 = d2_PM10_train.plot(ax=axes[r,c],legend=None,title='station'+str(count))
        # a3.set(xlabel=None)
        # a4 = d2_PM10_test.plot(ax=axes[r,c],legend=None,title='station'+str(count))
        # a4.set(xlabel=None)
        # splitting pm2.5 data
        data_PM25 = d2_PM25['df'+str(count)]
        d2_PM25_train = data_PM25[:24]
        d2_PM25_test = data_PM25[-12:]
        PM25_train["PM25_station"+str(count)] = d2_PM25_train
        PM25_test["PM25_station"+str(count)] = d2_PM25_test
        # ax =d2_PM25_train.plot(ax=axes[r,c],legend=None,title='station'+str(count))
        # ax.set(xlabel=None)
        # ax_2 =d2_PM25_test.plot(ax=axes[r,c],legend=None,title='station'+str(count))
        # ax_2.set(xlabel=None)
        count+=1
# plt.legend(bbox_to_anchor=(1.7,1.3))
```

```
# plt.show()

# Differencing PM10 and PM2.5 data
# non-seasonal differencing
# Dickey-Fuller Test Reference : https://www.statology.org/dickey-fuller-test-python/
# Get the results of test reference :https://www.analyticsvidhya.com/blog/2018/09/non-stationary-time-series-python/
nrow=5
ncol=5
fig, axes = plt.subplots(nrow, ncol)
stationary_test = []
stationary_test_format = []
result_test_PM10 = {}
adf_test_PM10 = {}
adf_test_PM10_2 = {}
adf_test_PM25 = {}
adf_test_PM25_2 = {}
station_test = {}
station_test_format = {}
diffed_PM10 = {}
diffed_PM25 = {}
count=101
for r in range(nrow):
    for c in range(ncol):
        # differencing all data
        st_diff = d2['df'+str(count)].diff()
        # st_diff.plot(ax=axes[r,c],legend=None,title='station'+str(count))
        print("station"+str(count)+" adf_check_PM10:\n")
        print(adfuller(st_diff['PM10'].dropna()))
        print("station"+str(count)+" adf_check_PM2.5:\n")
        print(adfuller(st_diff['PM2.5'].dropna()))

        station_test["PM10_station"+str(count)] = adfuller(st_diff['PM10'].dropna())
        station_test["PM2.5_station"+str(count)] = adfuller(st_diff['PM2.5'].dropna())
        stationary_test.append(station_test)

        result_PM10 = adfuller(st_diff['PM10'].dropna())
        result_fromat_PM10 = pd.Series(result_PM10[0:4], index=['Test Statistic','p-value','Lags Used','Number of Observations Used'])
        station_test_format["PM10_station"+str(count)] = result_fromat_PM10
        result_PM25 = adfuller(st_diff['PM2.5'].dropna())
```

```
        result_fromat_PM25 = pd.Series(result_PM25[0:4], index=['Test Statistic','p-value','Lags
Used','Number of Observations Used'])
        station_test_format["PM2.5_station"+str(count)] = result_fromat_PM25
        stationary_test_format.append(station_test_format)

        # differencing PM10 data (whole data)
        st_PM10_diff = d2_PM10['df'+str(count)].diff()

        st_PM10_diff.plot(ax=axes[r,c],legend=None,title='station'+str(count),xlabel='')
        # print("station"+str(count)+ adf_check_PM10:\n")
        # print(adfuller(st_PM10_diff.dropna()))
        adftest_PM10 = adfuller(st_PM10_diff.dropna())
        result_PM10 = pd.Series(adftest_PM10[0:4], index=['Test Statistic','p-value','Lags
Used','Number of Observations Used'])
        adf_test_PM10_2["station"+str(count)] = result_PM10
        # first differencing
        if adftest_PM10[1] > 0.05:
            # second differencing
            print("station"+str(count)+"non-stationary_1" + " " +"PM10, "+ str(adftest_PM10[1]))
            st_PM10_diff_2 = st_PM10_diff.diff().dropna()
            diffed_PM10["station"+str(count)] =st_PM10_diff_2
            adftest_PM10_2 = adfuller(st_PM10_diff_2.dropna())
            if adftest_PM10_2[1] > 0.05:
                print("station"+str(count)+"non-stationary_2" + " "+"PM10, "+ str(adftest_PM10_2[1])
+",cannot be modelled")
            else:
                print("station"+str(count)+"stationary_2" + " "+"PM10, " + str(adftest_PM10_2[1]))
                result_test_PM10 = pd.Series(adftest_PM10_2[0:4], index=['Test Statistic','p-value','Lags
Used','Number of Observations Used'])
                adf_test_PM10["station"+str(count)] = result_test_PM10
        else:
            diffed_PM10["station"+str(count)] =st_PM10_diff
            print("station"+str(count)+"one_diff_stationary "+ " " +"PM10, " + str(adftest_PM10[1]))
            result_test_PM10 = pd.Series(adftest_PM10[0:4], index=['Test Statistic','p-value','Lags
Used','Number of Observations Used'])
            adf_test_PM10["station"+str(count)] = result_test_PM10


        # differencing PM2.5 data
        st_PM25_diff = d2_PM25['df'+str(count)].diff()
        #diffed_PM25["station"+str(count)] =st_PM25_diff
```

```
        # st_PM25_diff.plot(ax=axes[r,c],legend=None,title='station'+str(count),xlabel='')
        # print("station"+str(count)+"adfcheck_PM2.5:\n")
        # print(adfuller(st_PM25_diff.dropna()))
        adftest_PM25 = adfuller(st_PM25_diff.dropna())
        result_PM25 = pd.Series(adftest_PM25[0:4], index=['Test Statistic','p-value','Lags
Used','Number of Observations Used'])
        adf_test_PM25_2["station"+str(count)] = result_PM25
        if adftest_PM25[1] > 0.05:
            print("station"+str(count)+"non-stationary_1" + " " +"PM2.5, "+ str(adftest_PM25[1]))
            st_PM25_diff_2 = st_PM25_diff.diff().dropna()
            diffed_PM25["station"+str(count)] =st_PM25_diff_2
            adftest_PM25_2 = adfuller(st_PM25_diff_2.dropna())
            if adftest_PM25_2[1] > 0.05:
                print("station"+str(count)+"non-stationary_2"+ " "    +"PM2.5, "+ str(adftest_PM25_2[1])
+",cannot be modelled")
            else:
                print("station"+str(count)+"stationary_2" + " "+"PM2.5, " + str(adftest_PM25_2[1]))
                result_test_PM25 = pd.Series(adftest_PM25_2[0:4], index=['Test Statistic','p-value','Lags
Used','Number of Observations Used'])
                adf_test_PM25["station"+str(count)] = result_test_PM25
        else:
            diffed_PM25["station"+str(count)] =st_PM25_diff
            print("station"+str(count)+"one_diff_stationary"+ " " +"PM2.5, " + str(adftest_PM25[1]))
            result_test_PM25 = pd.Series(adftest_PM25[0:4], index=['Test Statistic','p-value','Lags
Used','Number of Observations Used'])
            adf_test_PM25["station"+str(count)] = result_test_PM25
        count+=1
# plot p-value to check if it is less than 0.05
# if it is, then the time series is stable
PM10value_list=[]
PM25value_list=[]
count=101
for r in range(nrow):
    for c in range(ncol):
        value = adf_test_PM10_2['station'+str(count)].iloc[1]
        PM10value_list.append(value)
        value_2 = adf_test_PM25_2['station'+str(count)].iloc[1]
        PM25value_list.append(value_2)
        count+=1
b = np.arange(101, 126)
plt.scatter(b, PM25value_list) # To confirm the results, only the p-value of PM2.5 is plotted here
```

```
plt.title("Augmented Dickey-Fuller test : PM2.5")
plt.xlabel("Station")
plt.ylabel("p-value")
plt.show()
print("----------------------")
print("FINISH NON-SEASONAL DIFF")
print("----------------------")


# Seasonal differencing
# Repeat the above steps, only the parameter of diff() will change
# Dickey-Fuller Test Reference : https://www.statology.org/dickey-fuller-test-python/
# Get the results of test reference :https://www.analyticsvidhya.com/blog/2018/09/non-stationary-time-series-python/
nrow=5
ncol=5
fig, axes = plt.subplots(nrow, ncol)
ss_stationary_test = []
ss_stationary_test_format = []
ss_result_test_PM10 = {}
ss_adf_test_PM10 = {}
ss_adf_test_PM25 = {}
ss_adf_test_PM10_2 = {}
ss_adf_test_PM25_2 = {}
ss_station_test = {}
ss_station_test_format = {}
ss_adf_test_PM10 = {}
ss_diffed_PM10 = {}
ss_diffed_PM25 = {}
count=101
for r in range(nrow):
    for c in range(ncol):
        ss_PM10_diff = d2_PM10['df'+str(count)].diff(12)
        ss_PM10_diff.dropna().plot(ax=axes[r,c],title='station'+str(count),xlabel='')
        ss_adftest_PM10 = adfuller(ss_PM10_diff.dropna())
        result_PM10 = pd.Series(ss_adftest_PM10[0:4], index=['Test Statistic','p-value','Lags Used','Number of Observations Used'])
        ss_adf_test_PM10_2["station"+str(count)] = result_PM10
        # first differencing
        if adftest_PM10[1] > 0.05:
            # second differencing
```

```
            print("station"+str(count)+"non-stationary_1" + " " +"PM10, "+ str(ss_adftest_PM10[1]))
            ss_PM10_diff_2 = ss_PM10_diff.diff(12).dropna()
            ss_diffed_PM10["station"+str(count)] =ss_PM10_diff_2
            ss_adftest_PM10_2 = adfuller(ss_PM10_diff_2.dropna())
            if ss_adftest_PM10_2[1] > 0.05:
                print("station"+str(count)+"non-stationary_2" + " "+"PM10, "+ str(ss_adftest_PM10_2[1])
+",cannot be modelled")
            else:
                print("station"+str(count)+"stationary_2" + " "+"PM10, " + str(ss_adftest_PM10_2[1]))
                ss_result_test_PM10 = pd.Series(ss_adftest_PM10_2[0:4], index=['Test Statistic','p-
value','Lags Used','Number of Observations Used'])
                ss_adf_test_PM10["station"+str(count)] = ss_result_test_PM10
        else:
            ss_diffed_PM10["station"+str(count)] =ss_PM10_diff
            print("station"+str(count)+"stationary "+ " " +"PM10, " + str(ss_adftest_PM10[1]))
            ss_result_test_PM10 = pd.Series(ss_adftest_PM10[0:4], index=['Test Statistic','p-
value','Lags Used','Number of Observations Used'])
            ss_adf_test_PM10["station"+str(count)] = ss_result_test_PM10

        # differencing PM2.5 data
        ss_PM25_diff = d2_PM25['df'+str(count)].diff(12)
        ss_PM25_diff.dropna().plot(ax=axes[r,c],title='station'+str(count),xlabel='')
        ss_adftest_PM25 = adfuller(ss_PM25_diff.dropna())
        result_PM25 = pd.Series(ss_adftest_PM25[0:4], index=['Test Statistic','p-value','Lags
Used','Number of Observations Used'])
        ss_adf_test_PM25_2["station"+str(count)] = result_PM25
        if adftest_PM25[1] > 0.05:
            print("station"+str(count)+"non-stationary_1" + " " +"PM2.5, "+ str(ss_adftest_PM25[1]))
            ss_PM25_diff_2 = ss_PM25_diff.diff(12).dropna()
            ss_diffed_PM25["station"+str(count)] =ss_PM25_diff_2
            ss_adftest_PM25_2 = adfuller(ss_PM25_diff_2.dropna())
            if ss_adftest_PM25_2[1] > 0.05:

                print("station"+str(count)+"non-stationary_2"+ " "    +"PM2.5, "+
str(ss_adftest_PM25_2[1]) +",cannot be modelled")
            else:
                print("station"+str(count)+"stationary_2" + " "+"PM2.5, " + str(ss_adftest_PM25_2[1]))
                ss_result_test_PM25 = pd.Series(ss_adftest_PM25_2[0:4], index=['Test Statistic','p-
value','Lags Used','Number of Observations Used'])
                ss_adf_test_PM25["station"+str(count)] = ss_result_test_PM25
        else:
```

```
                ss_diffed_PM25["station"+str(count)] =ss_PM25_diff
                print("station"+str(count)+"stationary"+ " " +"PM2.5, " + str(ss_adftest_PM25[1]))
                ss_result_test_PM25 = pd.Series(ss_adftest_PM25[0:4], index=['Test Statistic','p-
value','Lags Used','Number of Observations Used'])
                ss_adf_test_PM25["station"+str(count)] = ss_result_test_PM25
            count+=1

# plot p-value
PM10value_list_2=[]
PM25value_list_2=[]
count=101
for r in range(nrow):
    for c in range(ncol):
        value = ss_adf_test_PM10_2['station'+str(count)].iloc[1]
        PM10value_list_2.append(value)
        value_2 = ss_adf_test_PM25_2['station'+str(count)].iloc[1]
        PM25value_list_2.append(value_2)
        count+=1
b = np.arange(101, 126)
plt.scatter(b, PM25value_list_2)
plt.title("Augmented Dickey-Fuller test : PM2.5")
plt.xlabel("Station")
plt.ylabel("p-value")
plt.show()
print("----------------------")
print("FINISH SEASONAL DIFF")
print("----------------------")

# plot acf
# non-seasonal diff
# reference: https://www.statsmodels.org/dev/generated/statsmodels.graphics.tsaplots.plot_acf.html
nrow=5
ncol=5
fig, axes = plt.subplots(nrow, ncol)
count=101
for r in range(nrow):
    for c in range(ncol):
        # plot acf of all PM10 data

sm.graphics.tsa.plot_acf(diffed_PM10['station'+str(count)].dropna().values.squeeze(),lags=20,ax=axes[r,
c])
```

```
        # plot acf of all PM2.5 data

sm.graphics.tsa.plot_acf(diffed_PM25['station'+str(count)].dropna().values.squeeze(),lags=20,ax=axes[r,
c],title='ACF:station'+str(count))
        count+=1

# plot pacf
# non-seasonal diff
# reference: https://www.statsmodels.org/dev/generated/statsmodels.graphics.tsaplots.plot_pacf.html
nrow=5
ncol=5
fig, axes = plt.subplots(nrow, ncol)
count=101
for r in range(nrow):
    for c in range(ncol): #non-seasonal diff
        # plot pacf of all PM10 data

sm.graphics.tsa.plot_pacf(diffed_PM10['station'+str(count)].dropna().values.squeeze(),ax=axes[r,c])
        # plot pacf of all PM2.5 data

sm.graphics.tsa.plot_pacf(diffed_PM25['station'+str(count)].dropna().values.squeeze(),ax=axes[r,c],title='
PACF:station'+str(count))
        count+=1

# plot acf
# seasonal diff
# reference: https://www.statsmodels.org/dev/generated/statsmodels.graphics.tsaplots.plot_acf.html
nrow=5
ncol=5
fig, axes = plt.subplots(nrow, ncol)
count=101
for r in range(nrow):
    for c in range(ncol):   #seasonal diff
        # plot acf of all PM10 data
sm.graphics.tsa.plot_acf(ss_diffed_PM10['station'+str(count)].dropna().values.squeeze(),lags=20,ax=axe
s[r,c])
        # plot acf of all PM2.5 data

sm.graphics.tsa.plot_acf(ss_diffed_PM25['station'+str(count)].dropna().values.squeeze(),lags=20,ax=axe
s[r,c],title='ACF:station'+str(count))
```

```
            count+=1

# plot pacf
# seasonal diff
# reference: https://www.statsmodels.org/dev/generated/statsmodels.graphics.tsaplots.plot_pacf.html
nrow=5
ncol=5
fig, axes = plt.subplots(nrow, ncol)
count=101
for r in range(nrow):
        for c in range(ncol): #seasonal diff
            # plot pacf of all PM10 data

sm.graphics.tsa.plot_pacf(ss_diffed_PM10['station'+str(count)].dropna().values.squeeze(),lags=11,ax=axes[r,c])
            # plot pacf of all PM2.5 data

sm.graphics.tsa.plot_pacf(ss_diffed_PM25['station'+str(count)].dropna().values.squeeze(),lags=11,ax=axes[r,c],title='PACF:station'+str(count))

            count+=1

#Repeat the above steps, but only draw the individual stations so no loops are needed
# pacf reference:
https://www.statsmodels.org/dev/generated/statsmodels.graphics.tsaplots.plot_pacf.html
# acf reference: https://www.statsmodels.org/dev/generated/statsmodels.graphics.tsaplots.plot_acf.html
# only plot acf in station112 PM10 data (non-seasonal diff)
sm.graphics.tsa.plot_acf(diffed_PM10['station'+str(112)].dropna().values.squeeze(),title='ACF : PM10, station'+str(112))
# only plot pacf in station112 PM10 data (non-seasonal diff)
sm.graphics.tsa.plot_pacf(diffed_PM10['station'+str(112)].dropna().values.squeeze(),title='PACF : PM10, station'+str(112))
# only plot acf in station112 PM10 data (seasonal diff)
sm.graphics.tsa.plot_acf(ss_diffed_PM10['station'+str(112)].diff(12).dropna().values.squeeze(),title='ACF(seasonal_diff) : PM10, station'+str(112))
# only plot pacf in station112 PM10 data (seasonal diff)
sm.graphics.tsa.plot_pacf(ss_diffed_PM10['station'+str(112)].diff(12).dropna().values.squeeze(),lags=5,title='PACF(seasonal_diff) : PM10, station'+str(112))
# only plot acf in station112 PM2.5 data (non-seasonal diff)
sm.graphics.tsa.plot_acf(diffed_PM25['station'+str(112)].dropna().values.squeeze(),title='ACF : PM2.5, station'+str(112))
```

```
# only plot pacf in station112 PM2.5 data (non-seasonal diff)
sm.graphics.tsa.plot_pacf(diffed_PM25['station'+str(112)].dropna().values.squeeze(),title='PACF : PM2.5, station'+str(112))
# only plot acf in station112 PM2.5 data (seasonal diff)
sm.graphics.tsa.plot_acf(ss_diffed_PM25['station'+str(112)].dropna().values.squeeze(),title='ACF(seasonal_diff) : PM2.5, station'+str(112))
# only plot pacf in station112 PM2.5 data (seasonal diff)
sm.graphics.tsa.plot_pacf(ss_diffed_PM25['station'+str(112)].dropna().values.squeeze(),lags=11,title='PACF(seasonal_diff) : PM2.5, station'+str(112))
# only plot acf in station116 PM2.5 data (non-seasonal diff)
sm.graphics.tsa.plot_acf(diffed_PM25['station'+str(116)].dropna().values.squeeze(),title='ACF : PM2.5, station'+str(116))
# only plot pacf in station116 PM2.5 data (non-seasonal diff)
sm.graphics.tsa.plot_pacf(diffed_PM25['station'+str(116)].dropna().values.squeeze(),title='PACF : PM2.5, station'+str(116))
# only plot acf in station116 PM2.5 data (seasonal diff)
sm.graphics.tsa.plot_acf(ss_diffed_PM25['station'+str(116)].dropna().values.squeeze(),title='ACF(seasonal_diff) : PM2.5, station'+str(116))
# only plot pacf in station116 PM2.5 data (seasonal diff)
sm.graphics.tsa.plot_pacf(ss_diffed_PM25['station'+str(116)].dropna().values.squeeze(),lags=11,title='PACF(seasonal_diff) : PM2.5, station'+str(116))
# only plot acf in station122 PM2.5 data (non-seasonal diff)
sm.graphics.tsa.plot_acf(diffed_PM25['station'+str(122)].dropna().values.squeeze(),title='ACF : PM2.5, station'+str(122))
# only plot pacf in station122 PM2.5 data (non-seasonal diff)
sm.graphics.tsa.plot_pacf(diffed_PM25['station'+str(122)].dropna().values.squeeze(),title='PACF : PM2.5, station'+str(122))
# only plot acf in station122 PM2.5 data (seasonal diff)
sm.graphics.tsa.plot_acf(ss_diffed_PM25['station'+str(122)].dropna().values.squeeze(),title='ACF(seasonal_diff) : PM2.5, station'+str(122))
# only plot pacf in station122 PM2.5 data (seasonal diff)
sm.graphics.tsa.plot_pacf(ss_diffed_PM25['station'+str(122)].dropna().values.squeeze(),lags=11,title='PACF(seasonal_diff) : PM2.5, station'+str(122))


#using auto_ARIMA to train PM10&PM2.5(station112,116,122) Data
pm10_train_112 = PM10_train["PM10_station"+str(112)]
pm10_test_112 = PM10_test["PM10_station"+str(112)]
pm10_train_116 = PM10_train["PM10_station"+str(116)]
pm10_test_116 = PM10_test["PM10_station"+str(116)]
```

```
pm10_train_122 = PM10_train["PM10_station"+str(122)]
pm10_test_122 = PM10_test["PM10_station"+str(122)]
pm25_train_112 = PM25_train["PM25_station"+str(112)]
pm25_test_112 = PM25_test["PM25_station"+str(112)]
pm25_train_116 = PM25_train["PM25_station"+str(116)]
pm25_test_116 = PM25_test["PM25_station"+str(116)]
pm25_train_122 = PM25_train["PM25_station"+str(122)]
pm25_test_122 = PM25_test["PM25_station"+str(122)]
# auto_ARIMA reference：https://www.alldatascience.com/time-series/forecasting-time-series-with-auto-arima/

# auto+Seasonal model - PM10
model_auto_pm10 = pm.auto_arima(pm10_train_112, start_p=1, start_q=1,
                                test='adf',
                                max_p=16, max_q=2,
                                m=12,
                                start_P=0,
                                D=1,d=1,
                                seasonal=True,
                                trace=True,
                                error_action='ignore',
                                suppress_warnings=True,
                                stepwise=True)
print(model_auto_pm10.summary()) #best: (1,1,0)x(0,1,0) #AIC:76.616 #Ljung-Box(L1):0.05
# plot residual erros
model_resid = model_auto_pm10.resid()
residuals = pd.DataFrame(model_resid)
fig = sm.graphics.tsa.plot_acf(residuals.values, lags=12) #residuals ACF
# residuals density plot reference: https://machinelearningmastery.com/visualize-time-series-residual-forecast-errors-with-python/
residuals.plot()
residuals.plot(kind='kde')
plt.show() # Confirmation that the model has passed the test (has removed the autocorrelation from the series) in order to predict
print(residuals.describe())

# Generate a forecast
fitted = model_auto_pm10.fit(pm10_train_112)
forecast    = fitted.predict(n_periods=12)
fc_series = pd.Series(forecast, index=pm10_test_112.index)
# plot the each data with different colors
```

```
# reference: https://frankiecancino.medium.com/forecasting-with-auto-arima-in-python-7d74bc61f0c3
plt.plot(pm10_train_112,label="training", c = 'black')
plt.plot(pm10_test_112,label="test", c = 'green')
plt.plot(fc_series,label="predicted", c = 'red')
# Set axis labels
plt.xlabel('Time')
plt.ylabel('Value')
plt.title("seasonal_diff, PM10 (station112)")
plt.show()


# auto+Non_seasonal model-PM10
model_auto_pm10 = pm.auto_arima(pm10_train_112, start_p=1, start_q=1,
                                test='adf',
                                max_p=3, max_q=3,
                                m=1,
                                d=None,
                                seasonal=False,
                                start_P=0,
                                D=0,
                                trace=True,
                                error_action='ignore',
                                suppress_warnings=True,
                                stepwise=True)
print(model_auto_pm10.summary()) #best: (1,0,0) AIC=175.969
# plot residual erros
model_resid = model_auto_pm10.resid()
residuals = pd.DataFrame(model_resid)
fig = sm.graphics.tsa.plot_acf(residuals.values, lags=12) #residuals ACF
plt.show() # Confirmation that the model has passed the test (has removed the autocorrelation from the
series) in order to predict
print(residuals.describe())
# Generate a forecast
fitted = model_auto_pm10.fit(pm10_train_112)
forecast    = fitted.predict(n_periods=12)
fc_series = pd.Series(forecast, index=pm10_test_112.index)
# plot the each data with different colors
# reference: https://frankiecancino.medium.com/forecasting-with-auto-arima-in-python-7d74bc61f0c3
plt.plot(pm10_train_112,label="training", c = 'black')
plt.plot(pm10_test_112,label="test", c = 'green')
plt.plot(fc_series,label="predicted", c = 'red')
```

```
# Set axis labels
plt.xlabel('Time')
plt.ylabel('Value')
plt.legend(bbox_to_anchor=(1.7,1.7))
plt.title("non_seasonal, PM10 (station112)")
plt.show()


#using auto ARIMA train PM2.5(station112) Data

# auto+Seasonal model - PM2.5
model_auto_pm25 = pm.auto_arima(pm25_train_112, start_p=1, start_q=1,
                                test='adf',
                                max_p=5, max_q=5,
                                m=12,
                                start_P=0,
                                D=1,d=1,
                                seasonal=True,
                                trace=True,
                                error_action='ignore',
                                suppress_warnings=True,
                                stepwise=True)
print(model_auto_pm25.summary()) #best:(2,1,0)x(0,1,0)
# plot residual erros
model_resid = model_auto_pm25.resid()
residuals = pd.DataFrame(model_resid)
fig = sm.graphics.tsa.plot_acf(residuals.values, lags=12) #residuals ACF
plt.show() # Confirmation that the model has passed the test (has removed the autocorrelation from the
series) in order to predict
print(residuals.describe())
# Generate a forecast
fitted_pm25 = model_auto_pm25.fit(pm25_train_112)
forecast_pm25    = fitted_pm25.predict(n_periods=12)
fc_series_pm25 = pd.Series(forecast_pm25, index=pm25_test_112.index)
# plot the each data with different colors
# reference: https://frankiecancino.medium.com/forecasting-with-auto-arima-in-python-7d74bc61f0c3
plt.plot(pm25_train_112,label="training", c = 'black')
plt.plot(pm25_test_112,label="test", c = 'green')
plt.plot(fc_series_pm25,label="predicted", c = 'red')
# Set axis labels
plt.xlabel('Time')
```

```
plt.ylabel('Value')
plt.title("seasonal_diff, PM2.5 (station112)")
plt.show()



# auto+Non_seasonal model-PM2.5
auto_pm25 = pm.auto_arima(pm25_train_112, start_p=1, start_q=1,
                          test='adf',
                          max_p=3, max_q=3,
                          m=1,
                          d=None,
                          seasonal=False,
                          start_P=0,
                          D=0,
                          trace=True,
                          error_action='ignore',
                          suppress_warnings=True,
                          stepwise=True)
print(auto_pm25.summary()) #best:(0,1,0)
# plot residual erros
model_resid = auto_pm25.resid()
residuals = pd.DataFrame(model_resid)
fig = sm.graphics.tsa.plot_acf(residuals.values, lags=12) #residuals ACF
plt.show() # Confirmation that the model has passed the test (has removed the autocorrelation from the
series) in order to predict
print(residuals.describe())
# Generate a forecast
fitted_pm25 = auto_pm25.fit(pm25_train_112)
forecast_pm25    = fitted_pm25.predict(n_periods=12)
fc_series_pm25 = pd.Series(forecast_pm25, index=pm25_test_112.index)
# plot the each data with different colors
# reference: https://frankiecancino.medium.com/forecasting-with-auto-arima-in-python-7d74bc61f0c3
plt.plot(pm25_train_112,label="training", c = 'black')
plt.plot(pm25_test_112,label="test", c = 'green')
plt.plot(fc_series_pm25,label="predicted", c = 'red')
# Set axis labels
plt.xlabel('Time')
plt.ylabel('Value')
plt.title("nonseasonal_diff, PM2.5 (station112)")
plt.show()
```

```
# Try ARIMA() in PM2.5(station112),and the result is the same as that of AUTO
# reference: https://machinelearningmastery.com/arima-for-time-series-forecasting-with-python/
pm25_model = ARIMA(pm25_train_112, order=(2,1,0),
seasonal_order=(0,1,0,12),enforce_stationarity=False)
# Generate a forecast
# reference: https://www.analyticsvidhya.com/blog/2020/10/how-to-create-an-arima-model-for-time-series-forecasting-in-python/
start = len(pm25_train_112)
end = len(pm25_train_112) + len(pm25_test_112) - 1
fitted_pm25 = pm25_model.fit(pm25_train_112)
forecast_pm25    = fitted_pm25.predict(start,end)
fc_series_pm25 = pd.Series(forecast_pm25, index=pm25_test_112.index)
# plot the each data with different colors
# reference: https://frankiecancino.medium.com/forecasting-with-auto-arima-in-python-7d74bc61f0c3
plt.plot(pm25_train_112,label="training", c = 'black')
plt.plot(pm25_test_112,label="test", c = 'green')
plt.plot(fc_series_pm25,label="predicted", c = 'red')
# Set axis labels
plt.xlabel('Time')
plt.ylabel('Value')
plt.title("ARIMA (seasonal), PM2.5 (station112)")
plt.show()
```

```
# Try ARIMA() in PM10(station112),and the result is the same as that of AUTO
# reference: https://machinelearningmastery.com/arima-for-time-series-forecasting-with-python/
pm10_model = ARIMA(pm10_train_112, order=(1,1,0),
seasonal_order=(0,1,0,12),enforce_stationarity=False)
# Generate a forecast
# reference: https://www.analyticsvidhya.com/blog/2020/10/how-to-create-an-arima-model-for-time-series-forecasting-in-python/
start = len(pm10_train_112)
end = len(pm10_train_112) + len(pm10_test_112) - 1
fitted_pm10 = pm10_model.fit(pm10_train_112)
forecast_pm10 = fitted_pm10.predict(start,end)
fc_series_pm10 = pd.Series(forecast_pm10, index=pm10_test_112.index)
# plot the each data with different colors
# reference: https://frankiecancino.medium.com/forecasting-with-auto-arima-in-python-7d74bc61f0c3
plt.plot(pm10_train_112,label="training", c = 'black')
```

```
plt.plot(pm10_test_112,label="test", c = 'green')
plt.plot(fc_series_pm10,label="predicted", c = 'red')
# Set axis labels
plt.xlabel('Time')
plt.ylabel('Value')
plt.title("ARIMA (seasonal), PM10 (station112)")
plt.show()


#auto+Seasonal model – PM10-st116 # Repeat the same steps as above
model_auto_pm10 = pm.auto_arima(pm10_train_116, start_p=1, start_q=1,
                                test='adf',
                                max_p=16, max_q=5,
                                m=12,
                                start_P=0,
                                D=1,d=1,
                                seasonal=True,
                                trace=True,
                                error_action='ignore',
                                suppress_warnings=True,
                                stepwise=True)
print(model_auto_pm10.summary()) #best: (0,1,0)x(0,1,0) #AIC:130.611 #Ljung-Box(L1):0.04
# plot residual erros
model_resid = model_auto_pm10.resid()
residuals = pd.DataFrame(model_resid)
fig = sm.graphics.tsa.plot_acf(residuals.values, lags=12) # residuals ACF
plt.show() # Confirmation that the model has passed the test (has removed the autocorrelation from the
series) in order to predict
print(residuals.describe())
# Generate a forecast
fitted = model_auto_pm10.fit(pm10_train_116)
forecast    = fitted.predict(n_periods=12)
fc_series = pd.Series(forecast, index=pm10_test_116.index)
# plot the each data with different colors
# reference: https://frankiecancino.medium.com/forecasting-with-auto-arima-in-python-7d74bc61f0c3
plt.plot(pm10_train_116,label="training", c = 'black')
plt.plot(pm10_test_116,label="test", c = 'green')
plt.plot(fc_series,label="predicted", c = 'red')
# Set axis labels
plt.xlabel('Time')
plt.ylabel('Value')
```

```
plt.title("seasonal_diff, PM10 (station116)")
plt.show()

#auto +Seasonal model - PM 2.5-st116
model_auto_pm25 = pm.auto_arima(pm25_train_116, start_p=1, start_q=1,
                                test='adf',
                                max_p=5, max_q=5,
                                m=12,
                                start_P=0,
                                D=1,d=1,
                                seasonal=True,
                                trace=True,
                                error_action='ignore',
                                suppress_warnings=True,
                                stepwise=True)
print(model_auto_pm25.summary()) #best:(0,1,0)x(0,1,0)
model_resid = model_auto_pm25.resid()
residuals = pd.DataFrame(model_resid)
fig = sm.graphics.tsa.plot_acf(residuals.values, lags=12) # residuals ACF
# Generate a forecast
fitted_pm25 = model_auto_pm25.fit(pm25_train_116)
forecast_pm25    = fitted_pm25.predict(n_periods=12)
fc_series_pm25 = pd.Series(forecast_pm25, index=pm25_test_116.index)
# plot the each data with different colors
# reference: https://frankiecancino.medium.com/forecasting-with-auto-arima-in-python-7d74bc61f0c3
plt.plot(pm25_train_116,label="training", c = 'black')
plt.plot(pm25_test_116,label="test", c = 'green')
plt.plot(fc_series_pm25,label="predicted", c = 'red')
# Set axis labels
plt.xlabel('Time')
plt.ylabel('Value')
plt.title("seasonal_diff, PM2.5 (station116)")
plt.show()

# auto+Seasonal model - PM 10-st122
model_auto_pm10 = pm.auto_arima(pm10_train_122, start_p=1, start_q=1,
                                test='adf',
                                max_p=16, max_q=5,
                                m=12,
                                start_P=1,
                                D=1,d=1,
```

```
                                seasonal=True,
                                trace=True,
                                error_action='ignore',
                                suppress_warnings=True,
                                stepwise=True)
print(model_auto_pm10.summary()) #best: (0,1,0)x(0,1,0) #AIC:109.973 #Ljung-Box(L1):0.04
# plot residual erros
model_resid = model_auto_pm10.resid()
residuals = pd.DataFrame(model_resid)
fig = sm.graphics.tsa.plot_acf(residuals.values, lags=12) # residuals ACF
plt.show() # Confirmation that the model has passed the test (has removed the autocorrelation from the
series) in order to predict
print(residuals.describe())
# Generate a forecast
fitted = model_auto_pm10.fit(pm10_train_122)
forecast    = fitted.predict(n_periods=12)
fc_series = pd.Series(forecast, index=pm10_test_122.index)
# plot the each data with different colors
# reference: https://frankiecancino.medium.com/forecasting-with-auto-arima-in-python-7d74bc61f0c3
plt.plot(pm10_train_122,label="training", c = 'black')
plt.plot(pm10_test_122,label="test", c = 'green')
plt.plot(fc_series,label="predicted", c = 'red')
# Set axis labels
plt.xlabel('Time')
plt.ylabel('Value')
plt.title("seasonal_diff, PM10 (station122)")
plt.show()

#auto +Seasonal model - PM 2.5-st122
model_auto_pm25 = pm.auto_arima(pm25_train_122, start_p=1, start_q=1,
                                test='adf',
                                max_p=5, max_q=5,
                                m=12,
                                start_P=0,
                                D=1,d=1,
                                seasonal=True,
                                trace=True,
                                error_action='ignore',
                                suppress_warnings=True,
                                stepwise=True)
print(model_auto_pm25.summary()) #best:(1,1,0)x(0,1,0)
```

```
model_resid = model_auto_pm25.resid()
residuals = pd.DataFrame(model_resid)
fig = sm.graphics.tsa.plot_acf(residuals.values, lags=12) # residuals ACF
# Generate a forecast
fitted_pm25 = model_auto_pm25.fit(pm25_train_122)
forecast_pm25    = fitted_pm25.predict(n_periods=12)
fc_series_pm25 = pd.Series(forecast_pm25, index=pm25_test_122.index)
# plot the each data with different colors
# reference: https://frankiecancino.medium.com/forecasting-with-auto-arima-in-python-7d74bc61f0c3
plt.plot(pm25_train_122,label="training", c = 'black')
plt.plot(pm25_test_122,label="test", c = 'green')
plt.plot(fc_series_pm25,label="predicted", c = 'red')
# Set axis labels
plt.xlabel('Time')
plt.ylabel('Value')
plt.title("seasonal_diff, PM2.5 (station122)")
plt.show()
```